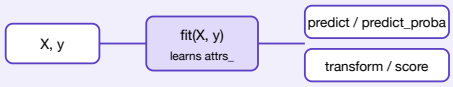
scikit-learn Cheat Sheet

scikit-learn 1.9 (1.9.1) · Python ≥ 3.11 · checked October 2026 · Irurueta Docs · original summary

1. The estimator API



Role	Methods
Estimator	`fit(X, y)` returns `self`
Predictor	`predict`, `predict_proba`, `decision_function`, `score`
Transformer	`fit_transform`, `transform`, `inverse_transform`, `get_feature_names_out`
Params	`get_params()`, `set_params(a__b=1)`, `clone(est)`

- Hyperparameters go to `__init__`; learned values end with `_` (`coef_`, `n_features_in_`). Unfitted use raises `NotFittedError`; check with `check_is_fitted`.
- X is 2-D (n_samples, n_features); y is 1-D. Pandas in, NumPy out unless `set_output(transform="pandas")` (or `set_config`).
- `FrozenEstimator(fitted)` (1.6): fit is a no-op, `clone` keeps the object; replaces `cv="prefit"`.
- Set `random_state` for anything random. 1.9 callbacks (`set_callbacks`, experimental).

2. Choosing an estimator

Goal	Try, in this order
Baseline	`DummyClassifier`, `DummyRegressor`
Category, small	`LogisticRegression`, `LinearSVC`, `KNeighborsClassifier`, `SVC`
Category, tabular	`HistGradientBoostingClassifier`, `RandomForestClassifier`
Category, text or 100K+	`MultinomialNB`, `SGDClassifier`
Quantity	`Ridge`, `Lasso`, `ElasticNet`, `HistGradientBoostingRegressor`, `SVR`
Groups	`KMeans`, `MiniBatchKMeans`, `GaussianMixture`, `DBSCAN`, `HDBSCAN`
Outliers	`IsolationForest`, `LocalOutlierFactor`
Reduce dimensions	`PCA`, `TruncatedSVD`; `TSNE` only to visualise

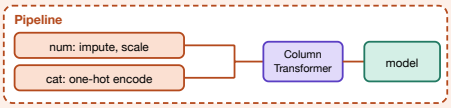
- Under about 50 samples: get more data. Fix metric and splitter before comparing models; same folds for all.
- Scaling matters for linear, SVM, KNN, PCA; trees and gradient boosting ignore it.
- `HistGradientBoosting*` accept `NaN` and detect pandas category columns (`categorical_features="from_dtype"`).

3. Preprocessing

Need	Class
Scale	`StandardScaler`, `MinMaxScaler`, `RobustScaler` (outliers)
Reshape distribution	`PowerTransformer`, `QuantileTransformer`, `FunctionTransformer(np.log1p)`
Nominal categories	`OneHotEncoder(handle_unknown="ignore", min_frequency=)`
Ordered categories	`OrdinalEncoder(categories=[[...]])`
High cardinality	`TargetEncoder(cv=)` (cross-fitted)
Missing values	`SimpleImputer(strategy=)`, `KNNImputer`, `add_indicator=True`
Bins, features	`KBinsDiscretizer`, `PolynomialFeatures`, `SplineTransformer`
Targets only	`LabelEncoder` (y), `TransformedTargetRegressor`
Select features	`SelectKBest`, `VarianceThreshold`, `RFECV`, `SelectFromModel`

- Fit every step on the training part only; leakage comes from fitting scalers or imputers before splitting.
- `OneHotEncoder` is sparse by default; `sparse_output=False` is needed for pandas output.

4. Pipeline and ColumnTransformer




```
num = make_pipeline(SimpleImputer(),
StandardScaler())
cat = OneHotEncoder(handle_unknown="ignore")
ct = ColumnTransformer([
  ("num", num,
make_column_selector(dtype_include="number")),
  ("cat", cat,
make_column_selector(dtype_include=
["category","str"]))],
  remainder="drop",
verbose_feature_names_out=False)
model = make_pipeline(ct,
LogisticRegression())
```

- Names: `make_pipeline` lower-cases class names; reach parameters as `logisticregression__C` or `columntransformer__num__simpleimputer__strategy`.
- `pipe[:-1]` slices steps; `named_steps["x"]`; `passthrough` or `"drop"` disable a step; `remainder="passthrough"` keeps others.
- `make_column_transformer`, `make_union`, `FeatureUnion`; `memory=` caches fits; `set_output(transform="pandas")`.
- Pipeline is a predictor when its last step is: search and cross-validate the whole pipeline, never pre-transformed data.
- Select columns by name, dtype (`make_column_selector`) or position; unseen categories at predict time need `handle_unknown="ignore"`.

5. Splitters and search

Data	Splitter
Hold-out	`train_test_split(X, y, test_size=.25, stratify=y, random_state=0)`
i.i.d.	`KFold`, `ShuffleSplit`, `RepeatedKFold`
Class imbalance	`StratifiedKFold(5, shuffle=True, random_state=0)`
Groups (patients)	`GroupKFold`, `StratifiedGroupKFold`, `GroupShuffleSplit`
Time	`TimeSeriesSplit(n_splits=5, gap=)`
Tiny sets	`LeaveOneOut`, `RepeatedStratifiedKFold`

Function	Use
`cross_val_score(m, X, y, cv=, scoring=)`	one score per fold
`cross_validate(m, X, y, scoring=[..], return_train_score=True)`	times, several metrics
`cross_val_predict(m, X, y, method="predict_proba")`	out-of-fold predictions
`GridSearchCV`, `RandomizedSearchCV(n_iter=)`	tune; `refit=True`
`HalvingGridSearchCV` (experimental import)	successive halving
`learning_curve`, `validation_curve`	bias or variance diagnosis
`TunedThresholdClassifierCV(m, scoring="f1")`	tune the decision threshold

- `best_estimator_`, `best_params_`, `cv_results_`. Nested CV for an honest estimate after tuning. Keep a test set untouched.

6. Metrics and Displays

Task	`scoring=` string
Classify	`accuracy`, `balanced_accuracy`, `f1`, `f1_macro`, `precision`, `recall`
Ranking	`roc_auc`, `roc_auc_ovr`, `average_precision`
Probabilities	`neg_log_loss`, `neg_brier_score`
Regress	`r2`, `neg_root_mean_squared_error`, `neg_mean_absolute_error`, `neg_mean_absolute_percentage_error`

- Scorers are "greater is better", so errors are negated. List all: `get_scorer_names()`. Custom: `make_scorer(f, greater_is_better=False)`.
- Functions: `classification_report`, `confusion_matrix`, `roc_curve`, `precision_recall_curve`, `matthews_corrcoef`, `root_mean_squared_error`, `d2_brier_score`, `top_k_accuracy_score`. Multi-class: `average="macro"` or `"weighted"`.
- 1.8 / 1.9: `confusion_matrix_at_thresholds`, `metric_at_thresholds`.

Displays (Matplotlib): `ConfusionMatrixDisplay`, `RocCurveDisplay`, `PrecisionRecallDisplay`, `DetCurveDisplay`, `PredictionErrorDisplay`, `CalibrationDisplay`, `LearningCurveDisplay`, `ValidationCurveDisplay`, `PartialDependenceDisplay`.

```
RocCurveDisplay.from_estimator(m, X_te, y_te,
ax=ax)
ConfusionMatrixDisplay.from_predictions(y_te,
pred, normalize="true")
RocCurveDisplay.from_cv_results(cv_res, X, y)
# 1.7+
```

7. Saving models

Format	Use when
`joblib.dump(m, "m.joblib")`	trusted, same versions; fast with big arrays
`pickle`, `cloudpickle`	trusted; custom functions in pipelines
`skops.io.dump` / `load(trusted=)`	untrusted source: lists untrusted types first
ONNX (`skl2onnx`)	serve elsewhere, no Python needed

- Loading a pickle or joblib file can run arbitrary code: only open files you made. A different version may load but warn (`InconsistentVersionWarning`) or break: pin versions, save a `requirements` file with the model.
- Save the whole Pipeline, not just the model, so preprocessing travels with it.

8. Renamed, removed, deprecated

Old	Now
`plot_roc_curve`, `plot_confusion_matrix` (rm 1.2)	`RocCurveDisplay.from_estimator`
`OneHotEncoder(sparse=)` (rm 1.4)	`sparse_output=`
`get_feature_names()`	`get_feature_names_out()`
`mean_squared_error(squared=False)`	`root_mean_squared_error`
`load_boston` (rm 1.2)	`fetch_california_housing`
`sklearn.cross_validation`, `grid_search`	`sklearn.model_selection`
`Imputer`	`SimpleImputer`
`CalibratedClassifierCV(cv="prefit")`	`FrozenEstimator`
`LogisticRegression(penalty=)` (dep 1.8, rm 1.10)	`l1_ratio=`, `C=`
`SVC(probability=True)` (dep 1.9)	`CalibratedClassifierCV(SVC(), ensemble=False)`
`PassiveAggressive*` (dep 1.8)	`SGDClassifier`
`TargetEncoder(shuffle=)` (dep 1.9)	`cv=KFold(5, shuffle=True)`

- Deprecations last two minors: run tests with `-W error::FutureWarning`. New hard dependency: narwhals (1.9). `IterativeImputer` still needs the `enable_iterative_imputer` import on 1.9.

Changed since older tutorials: Display classes replaced the `plot_*` functions · `sparse_output`, `get_feature_names_out`, `root_mean_squared_error` · no `load_boston`, `Imputer`, `sklearn.cross_validation` · `FrozenEstimator` replaces `cv="prefit"` · `HistGradientBoosting*` handle `NaN` and categories natively · `set_output` keeps DataFrames · metadata routing and callbacks are new · 1.10 is upcoming, not released.

Original summary, verified by running examples on scikit-learn 1.9.1 / Python 3.13. Full guide: albertoirurueta.github.io/docs → Data Science → scikit-learn.